CoT Quality Signals Are Domain-Conditioned:

A Measurement Perspective on Math, Science, and Coding

Yuhan Chi
Fudan University
masterwuguicyh@gmail.com

Abstract

Process supervision and verifier-based reranking rest on an implicit assumption: that chain-of-thought (CoT) traces encode answer quality in a domain-neutral way. We test that assumption by using a small interpretable probe—not a new verifier—as a measurement instrument. In math and science, the signal is strong: AUC-of-AUROC reaches 0.958 on math and 0.799 on science, with near-lossless frozen-basis transfer across anchor positions (ΔAUROC $= -0.001$ forward for math). In coding, the same feature family fails to exceed a 0.506 token-confidence baseline (AUC-of-AUROC 0.434), and a broader sweep over 102 coding-side scalars finds only modest alternatives. A one-hidden-layer MLP on the same feature cache preserves the ordering: 0.941/0.723/0.497 for math/science/coding.

This is not a claim that all verifiers fail on coding. It is a narrower, better-supported claim: within this interpretable feature family, the signal *stops measuring the same thing* once the domain changes. A small validation study of local state breaks is consistent with that picture—math break-like events coexist with recovery, while coding ones appear entangled with unstable execution state—but the main result stands on the full-scale comparison and does not depend on the annotation studies. Reproducible scripts and tables are in `scripts/` and `results/tables/`.

1 Introduction

There is a straightforward story about CoT-based verifiers: longer, more reflective traces tend to be more correct, so you can use trace features as a cheap quality proxy and use them to pick the best candidate among multiple rollouts. This story is well-supported for mathematical reasoning [3–5]. The question is whether it generalizes.

Our starting point is *not* a new architecture. We use a fixed three-stage pipeline—`StandardScaler` → `TruncatedSVD` → `LogisticRegression`—trained separately on three domains: math competitions (AIME24/25, BRUMO25, HMMT25), science (GPQA), and coding (LiveCodeBench-v5). The probe is simple enough that a performance failure can be attributed to the feature family, not to model capacity.

The central finding is a measurement breakdown. When the same feature family is applied to coding traces, it not only weakens—it falls *below* the baseline provided by raw token confidence. That ordering survives replacing the linear head with a nonlinear one, and it survives a broad sweep over alternative coding-side features. We interpret this as the signal no longer measuring answer quality in coding the way it does in math.

We add a small annotation study on *local state breaks*—moments where the reasoning state appears to fragment or contradict itself—to probe one candidate mechanism. The study is exploratory and has real validity limits (detailed in Section 6). It is not what drives the main finding.

2 Related Work

The core literature on CoT reasoning [1, 2] established that traces carry information beyond the final answer. Learned outcome and process reward models built on that insight [3, 4], with recent extensions adding automatic step-level supervision, cross-model benchmarks, and generative verifier formulations [5–7].

Our work sits closer to two adjacent strands. First, faithfulness analyses ask whether CoT traces mean what they appear to mean [8]; our measurement framing is in the same spirit. Second, intrinsic-self-correction work documents that reflection-like text does not always improve reasoning [9–12]. We add a cross-domain dimension: not only is self-correction unreliable on average, but the *measurability* of quality from trace features is itself domain-conditioned.

A useful analogy comes from psychometric measurement invariance [13]: the same instrument can measure different latent constructs in different populations. Probing work in NLP has made a related point about representation analysis [14]: a probe's performance depends on what the representation encodes, not just on the probe's capacity. Our result extends that logic to the verifier setting—feature families that probe correctness in math

may not probe anything interpretable in coding.

3 Setup

3.1 Domains and metrics

We use three data regimes from the repository: **Math:** 7,680 aligned runs across AIME24, AIME25, BRUMO25, and HMMT25. **Science:** 12,672 runs from GPQA. **Coding:** 10,688 runs from LiveCodeBench-v5.

Each run is observed at four anchor positions $a \in \{10, 40, 70, 100\}$, corresponding to the visible fraction of the trace. We report two summary metrics: **AUROC@100%:** final-trace discrimination. **AUC-of-AUROC:** area under the anchor-wise AUROC curve, used as an early-stopping summary.

For cross-anchor transfer, we freeze the low-rank basis learned at a source anchor and refit only the logistic head at the target:

$$\Delta\text{AUROC} = \text{AUROC}_{\text{frozen}} - \text{AUROC}_{\text{task-specific}}.$$

Values near zero mean the source basis already captures a reusable geometry.

3.2 Feature probe

The probe uses three families of cached features, intentionally kept cheap to isolate the domain effect from model capacity:

Token-confidence summaries. `tok_conf_prefix` is the mean $-\log p$ over the model's top-token distribution across visible prefix tokens.

Trajectory summaries. `traj_continuity` is the mean Jaccard similarity between consecutive 32-token activation slices; `traj_novelty` is mean $1 -$ max similarity to any prior slice; `traj_reflection_count` is the negated count of non-adjacent slice revisits exceeding similarity threshold 0.30.

Neuron-summary. `nc_mean` is the prefix mean row width of the sparse rows-bank—the average number of active keys per visible slice—derived from the neuron activation cache rather than from text.

The probe is fit separately per domain. The question is not whether a universal verifier exists; it is whether the *meaning* of this feature family stays stable enough within a domain and whether it changes across domains.

3.3 Validity note on the annotation study

The local-state-break annotation study (Section 5) is exploratory. On the math side, model-assisted labeling with author review of sampled outputs yields $\kappa = 0.603$ on a 30-run human validation sample—acceptable for a pilot.

Domain	AoA	A@100	Fwd Δ	Bwd Δ
Math	**.958**	**.982**	−.001	−.006
Science	**.799**	**.841**	−.010	−.001
Coding	.434	.407	—	—
Baselines (all domains, no anchor dependence)				
`tok_conf`	.506/.809/.506	.490/.856/.490	—	—
SC oracle†	.953/.921/.961	.953/.921/.961	—	—

Table 1: Main holdout result. AoA = AUC-of-AUROC; A@100 = AUROC at 100% anchor. Values for baselines are math/science/coding. Frozen-basis ΔAUROC is relative to a task-specific basis at the target anchor. †SC oracle = self-consistency upper bound using leave-one-out agreement rate (requires ground-truth labels; not achievable at inference time).

On the coding side, the original GLM-assisted protocol yields $\kappa = 0.0$ on the same 30-run check (the model marks 100% of runs positive; humans mark 6.7%). A replacement execution-semantic protocol gives $\kappa = 0.329$ across $n = 56$ items (below threshold). These numbers mean the coding annotation study *cannot* support prevalence claims or fine-grained taxonomy. We use it only for the protocol-separability question: does the same annotation logic transfer across domains?

4 Math and Science: Strong, Temporally Structured, Transferable

Table 1 shows the main result. The same probe is strong in math, respectable in science, and poor in coding.

Math is not just a final-score story—it is a temporal-structure story. Across the aligned bridge table, `traj_reflection_count` strengthens steadily as more of the trace is revealed:

$$\rho = -0.345 \text{ at } 10\%, \quad -0.432 \text{ at } 40\%, \quad -0.551 \text{ at } 100\%.$$

Early reflection and late reflection mean different things. By the end of a trace, persistent reflection is a strong failure predictor. That is the signature of a process feature, not a surface feature.

Science is weaker and less symmetric. The forward transfer gap (−0.010) is noticeably larger than the backward gap (−0.001), suggesting that science reasoning matures later and less uniformly. The feature geometry stabilizes on a different schedule—consistent with the measurement framing: the same probe tracks a different process in a different domain.

A one-hidden-layer MLP on the same 25-dimensional feature cache under five-fold GroupKFold by problem gives 0.941/0.723/0.497 mean anchor AUROC for math/science/coding; the 100%-anchor AUROCs are

0.968/0.755/0.507. Changing the classifier does not rescue coding. The cross-domain pattern is not a linear-head artifact.

5 Coding: The Signal Breaks Down

5.1 Full-scale result

On the full 10,688-run LiveCodeBench table, the best lexical proxy for self-corrective control-flow turbulence—`condition_reversal_per1k`—is statistically nonzero:

$$\rho = -0.062, \qquad p = 1.15 \times 10^{-10}.$$

But the probe that works in math yields AUC-of-AUROC 0.434 here, which is below a single-feature token-confidence baseline (0.506). The signal exists in the features but it does not separate correct from incorrect runs usefully.

We screen 102 interpretable coding-side scalars. The strongest negative scalar is `struct_density` ($\rho = -0.110$, oriented AUROC 0.563), followed by `cot_trace_count` ($\rho = -0.099$, AUROC 0.561). The strongest program-structure count, `cf_n_ifs`, has $\rho = +0.109$ (AUROC 0.565)—positive rather than negative. More complex code is weakly associated with more correct answers, not less. The feature sweep does not support a monotone "more complex → more wrong" story. Combining the top four interpretable scalars still reaches only AUROC 0.556 under GroupKFold by problem.

This is the right place to be explicit about scope. We do not claim that all stronger verifiers fail on coding. Code-execution-aware verifiers, models with access to compiler feedback, or probes over hidden states have different information and we make no claim about them. The finding is specific: within this feature family, the measurement breaks down in coding.

5.2 Annotation study: protocol separability

Because the coding GLM annotation was invalid ($\kappa = 0.0$; Section 3.3), we ran a replacement execution-semantic protocol on a partial LiveCodeBench subset. Table 2 reports the expanded math pilot alongside the coding comparison.

The prevalence of local state breaks is not discriminative in math ($p = 0.893$), so we do not use break frequency as a positive math claim. What the study does show is a geometry contrast. Inside the break-positive subset (78 correct / 63 incorrect math runs; 36 correct / 39 incorrect coding runs), the features that work in math are flat in coding:

- `traj_reflection_count` at 100%: math $d = 1.008$ (95% CI $[0.58, 1.48]$); coding $d = 0.013$ ($[-0.77, 0.87]$).

Quantity	Value	Note
Pilot-selected math runs	256	expanded sample
Usable parsed runs	245	after parse checks
Verified spans	221	stage-2 approved
Correct with break	84/128	rate = 0.656
Incorrect with break	78/117	rate = 0.667
Fisher exact p	0.893	not significant
Execution-break coding protocol ($\kappa = 0.329$, $n = 56$)		
GLM coding human κ	0.0	original protocol, invalid
Replacement κ	0.329	below threshold

Table 2: Expanded math pilot (break prevalence) and coding annotation validity summary. Break prevalence in math is not discriminative and is not used as a main claim.

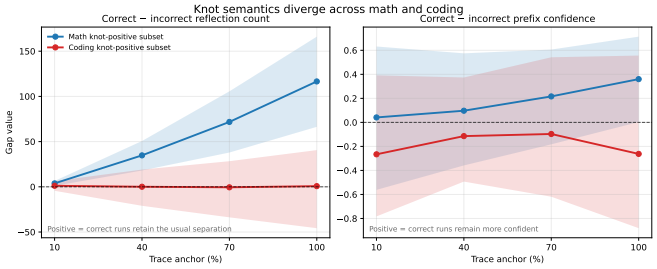

Figure 1: Feature gap inside break-positive subsets (correct minus incorrect mean, with 95% cluster-bootstrap bands). Left: `traj_reflection_count`. Right: `tok_conf_prefix`. Math trends up; coding stays flat.

- `tok_conf_prefix` at 100%: math $d = 0.392$ ($[0.01, 0.82]$); coding $d = -0.284$ ($[-1.00, 0.67]$).
- `nc_mean` at 100%: math $d = 0.989$ ($[0.59, 1.44]$); coding $d = -0.141$ ($[-0.94, 0.61]$).

In the expanded math pilot, the correct-minus-incorrect reflection gap is already positive at 10% anchor (95% CI $[1.07, 6.47]$) and grows to $[66.34, 166.10]$ at 100%. The corresponding coding gap stays near zero at every anchor.

Figure 1 shows this directly. The y-axis is $\mathbb{E}[\text{feature} \mid \text{correct, break-positive}] - \mathbb{E}[\text{feature} \mid \text{incorrect, break-positive}]$. In math the gaps trend in the expected direction. In coding they stay near zero or reverse.

A 240-run GLM-assisted coding audit gives the same reading: knot severity correlates $\rho = -0.023$ with correctness ($p = 0.723$); knot-present accuracy is 0.537 vs. 0.545 when absent; and 228 of 240 runs fall into `lost_state`. The labels are easy to elicit but do not separate correct from incorrect runs—which is exactly what the measurement framing predicts. If the feature no longer tracks the quality object, annotating it more carefully will not fix the probe.

The timing extension uses 450 already-labeled examples (221 math, 229 coding). Onset localization succeeds in 272 cases. Among localized coding cases, all 125 fall in the first third of the trace (mean normalized onset 0.032). Math has a longer tail: mean onset 0.109, with 132 early / 11 middle / 4 late. The localization only succeeds on a subset, so this is suggestive rather than conclusive. But it fits: coding failures appear early as state-serialization trouble; math explicit breaks can emerge later as proof state is re-bound or contradicted.

6 Limitations and Conclusion

Several limits deserve direct statement.

First, the probe is deliberately simple. We do not claim state-of-the-art verifier accuracy and we do not compare against hidden-state probes, execution-aware verifiers, or generative verifier architectures. A small MLP sanity check reduces the linear-head-artifact risk but is not a baseline competition.

Second, the annotation study has real validity problems. The original coding GLM protocol yielded $\kappa = 0.0$ and is invalid for any inferential purpose. The replacement execution-semantic protocol reached $\kappa = 0.329$ ($n = 56$), which is below acceptable threshold. These numbers are disclosed here because they constrain interpretation: the coding annotation study supports a protocol-separability observation, nothing more. All main results depend on the full-scale quantitative comparison in Table 1, not on the annotation studies.

Third, the RL regime is intentionally excluded. Verifier drift under RL fine-tuning is a real phenomenon in the repository (FPR rising from 0.982 to 0.998 across training steps), but it fits a different paper axis—regime conditioning rather than domain conditioning—and is being written separately.

The main conclusion follows from what survives all three limits:

> **CoT quality signals are domain-conditioned.** In math and science, an interpretable trace probe can be strong, early-onset, and transferable across anchor positions. In coding, the same feature family drops below a token-confidence baseline and is not rescued by a broader feature sweep or a nonlinear classifier. The most plausible account is a measurement breakdown: code correctness depends on executable state evolution, whereas text and trajectory features observe only an imperfect shadow of that object.

The practical implication is simple: before deploying a CoT-based verifier in a new domain, check whether the feature family measures the same thing there. Predictive performance in math does not transfer as a reliability guarantee to coding.

Artifacts. Quantitative claims reproduce from `results/tables/`. Analysis scripts are in `scripts/`. The paper source is in `workshop/`.

References

[1] Jason Wei et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

[2] Xuezhi Wang et al. Self-consistency improves chain of thought reasoning in language models. In *ICLR*, 2023.

[3] Karl Cobbe et al. Training verifiers to solve math word problems. *arXiv:2110.14168*, 2021.

[4] Hunter Lightman et al. Let's verify step by step. *arXiv:2305.20050*, 2023.

[5] Peiyi Wang et al. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *ACL*, 2024.

[6] Chujie Zheng et al. ProcessBench: Identifying process errors in mathematical reasoning. In *ACL*, 2025.

[7] Lunjun Zhang et al. Generative verifiers: Reward modeling as next-token prediction. In *ICLR*, 2025.

[8] Tamas K. Feher de Lanham et al. Measuring faithfulness in chain-of-thought reasoning. *arXiv:2307.13702*, 2023.

[9] Dancheng Liu et al. Large language models have intrinsic self-correction ability. In *NeurIPS*, 2024.

[10] Aman Madaan et al. Self-Refine: Iterative refinement with self-feedback. *arXiv:2303.17651*, 2023.

[11] Noah Shinn et al. Reflexion: Language agents with verbal reinforcement learning. *arXiv:2303.11366*, 2023.

[12] Jie Huang et al. Large language models cannot self-correct reasoning yet. *arXiv:2310.01798*, 2023.

[13] Robert E. Vandenberg and Charles E. Lance. A review and synthesis of the measurement invariance literature: Suggestions, practices, and recommendations for organizational research. *Organizational Research Methods*, 3(1):4–70, 2000.

[14] Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022.